QUESTION

What is the difference between early termination and pruning?

QUESTION

Why does a recursive search using `||` stop after finding a solution?

QUESTION

What is the key rule for safe pruning?

QUESTION

When is the condition `rem < 0` safe for subset-sum search?

QUESTION

Why is `rem < 0` unsafe when negative numbers are allowed?

QUESTION

What does `suffix[k]` represent?

ANSWER

C++ uses **short-circuit evaluation**. When the first recursive call in `a || b` returns `true`, the second call is **not evaluated**. The `true` then propagates upward: each active call returns `true` to its caller, so the whole search stops.

ANSWER

Early termination stops the entire search after finding a satisfactory solution.

Pruning skips a branch that cannot help: either it cannot succeed (**infeasibility pruning**), or exploring it would add nothing new (**duplicate-state pruning**).

ANSWER

When all remaining numbers are **nonnegative**. Adding more of them cannot bring a negative remainder back to zero.

ANSWER

Never discard a branch unless you can **justify** it.

Infeasibility pruning: the branch cannot contain a valid solution.

Duplicate-state pruning: an equivalent state has already been explored, so this branch adds no new outcome that the search needs.

ANSWER

The sum of all remaining numbers `v[k]` through `v[n-1]`, with `suffix[n] = 0`.

ANSWER

A negative number might bring the remainder back to zero.

Example: remaining numbers `{5, -3}`, target 2. Taking 5 makes `rem = -3`, but taking `-3` afterward succeeds.

QUESTION

Why is `rem > suffix[k]` a safe pruning condition?

QUESTION

What are the three main stopping conditions in the bounded subset-sum search?

QUESTION

For `v = {5,4,3,2,1}`, what is `suffix[0..5]`?

QUESTION

For `v = {5,4,3,2,1}`, why does target 20 take only one call?

QUESTION

What is one state in the pair-selection Make 24 search?

QUESTION

Why does the Make 24 search choose pairs with `i < j`?

ANSWER

1. Goal: `rem == 0` → return `true`.

2. No numbers remain: `k == n` → return `false`.

3. Pruning: `rem < 0 || rem > suffix[k]` → return `false`.

The pruning tests assume all remaining numbers are nonnegative.

ANSWER

For nonnegative numbers, even taking **every** remaining number cannot reach `rem` if their total sum is too small.

ANSWER

Because `rem = 20` exceeds `suffix[0] = 15`. The root call immediately rejects the branch.

ANSWER

`{15, 10, 6, 3, 1, 0}`

ANSWER

It chooses each **unordered pair once**, avoiding a redundant second selection of the same two entries.

ANSWER

The collection of numbers currently on the table. A move replaces two selected numbers with one result, reducing the number of entries by one.

QUESTION

What six operation choices are tried for a selected pair `a, b`?

QUESTION

Why are both subtraction and division orders needed?

QUESTION

How many first moves are possible with four numbers, assuming all six operations?

QUESTION

Assuming all six operations are available at every step, how many complete candidate operation sequences are there for four numbers?

QUESTION

What is a duplicate state in Make 24?

QUESTION

Why sort the values in a duplicate-state key?

ANSWER

They are generally **noncommutative**.

In contrast, reversing the operands of addition or multiplication does not change the result.

ANSWER

`a+b`, `a*b`, `a-b`, `b-a`, `a/b`, and `b/a`.

Division choices are skipped when their denominators are zero or too close to zero.

ANSWER

$36 \times 18 \times 6 = \mathbf{3{,}888}$ candidate sequences, counted **before** invalid-division filtering, duplicate elimination, or early termination.

An actual solver may explore fewer.

ANSWER

$\binom{4}{2} \times 6 = 6 \times 6 = \mathbf{36}$.

ANSWER

To make states that differ only in the order of their remaining numbers use the **same canonical key**.

ANSWER

A state containing the same remaining numerical values as a previously explored state, even if those values appear in a **different order**.

Scope: for deciding whether a solution exists (or finding one solution), such states are interchangeable, assuming the same operations are available. To enumerate every distinct expression, retain distinct expression histories; do not discard them solely because their numerical states match.

QUESTION

What does `seen.insert(key).second` mean for a `std::set`?

QUESTION

Why can rounded floating-point state keys be unsafe?

QUESTION

When can a sorted integer state key represent duplicates exactly?

QUESTION

Why is returning `true` before undo sometimes safe?

QUESTION

What does `printUnique` use to avoid repeated permutations?

QUESTION

Why must the `tried` set be local to each recursive call?

ANSWER

Two different numerical states may round to the same key. Skipping one of them could **discard a valid solution**.

ANSWER

It is `true` when the key was **newly inserted**, and `false` when the key was **already present**.

ANSWER

It is safe when the modified state does not need to be restored, such as a **local vector** in the Make 24 solver.

It is not automatically safe for shared state that callers expect unchanged.

ANSWER

When the states use exact integer values and the key preserves those values without rounding or loss of precision.

ANSWER

A character may be used at different positions, and different calls at the same depth explore different prefixes.

Skip repeated character choices only within **this call's** candidate loop, and create a **fresh set for each call**.

ANSWER

A **local set** of characters tried at the current recursive position. It skips a character value already tried at that position.

QUESTION

How many distinct permutations does `AABB` have?

QUESTION

How many lines do `printAll` and `printUnique` generate for `LEVEL`?

QUESTION

Must `v` be sorted for the take/skip subset-sum search?

QUESTION

For `v = {5, 1, 2}` and target 4 (`suffix = {8, 3, 2, 0}`), how many calls are made with and without pruning?

QUESTION

For `v = {9,8,...,1}` and target 20, how many calls does the search make with (a) no savings (full traversal), (b) early termination only, (c) early termination and `rem < 0`?

QUESTION

In Make 24 Version 1 (ordered pairs, 4 operations), why are there $12 \times 4 = 48$ first moves with four numbers?

ANSWER

`printAll` prints $5! = \mathbf{120}$ lines.

`printUnique` prints $\dfrac{5!}{2!\,2!} = \mathbf{30}$ distinct arrangements.

ANSWER

$\dfrac{4!}{2!\,2!} = \mathbf{6}$.

ANSWER

Without any pruning:
$1 + 2 + 4 + 8 = 2^4 - 1 = \mathbf{15}$ calls.

With both tests: **3** calls.

1. Root: `rem = 4`.

2. Take 5: `rem = -1`, stopped by `rem < 0`.

3. Skip 5: `rem = 4 > suffix[1] = 3`, stopped by the bound.

ANSWER

No. The search follows the order of `v`.

For `v = {4, 1, 2, 3}` and target 6, it makes **9 calls** and finds `chosen = {4, 2}`.

Assumptions for the count of 9: the search tries **take before skip**, stops at the first success (`rem == 0`), and includes the `rem < 0` test. Without the `rem < 0` test it makes **11** calls. The subset `{1, 2, 3}` also works, but `{4, 2}` is found first.

Sorting is not needed. Only pruning tests such as `rem < 0` need nonnegative numbers.

ANSWER

Think of **friends**: 4 numbers, and each number has 3 friends, so $4 \times 3 = 12$ ordered pairs (x, y). Each pair gets 4 operations $(+, -, \times, \div)$: $12 \times 4 = \mathbf{48}$.

The 4 in "×4" counts **operations**, not numbers. With 3 numbers left: $3 \times 2 \times 4 = 24$. With 2 left: $2 \times 1 \times 4 = 8$. Total: $48 \times 24 \times 8 = \mathbf{9{,}216}$ candidate sequences, counted before invalid-division filtering, duplicate elimination, or early termination.

ANSWER

(a) **1023**. (b) **244**. (c) **12**.

These counts assume **take before skip** and count every function entry, including the root and immediately rejected calls. "No savings" means full traversal: no early stop on success and no pruning.

The 12 calls reach the solution `{9, 8, 3}`. Without the `rem < 0` test, four overshooting calls keep going: $12 + 14 + 30 + 62 + 126 = \mathbf{244}$, where $14, 30, 62, 126$ are $2^k - 2$ for $k = 4, 5, 6, 7$.

QUESTION

In the Make 24 solver, how is `rest` built for the pair at positions `i` and `j`?

QUESTION

For each result `r`, in what order do we push, test and pop on `rest`?

QUESTION

Why test `std::abs(nums[0] - 24) < 1e-9` instead of `nums[0] == 24`?

QUESTION

How is the key `vector<long long>` built for a state, and why `llround(x * 1e6)`?

QUESTION

Why is `return false` correct when `seen.insert(key).second` is `false`?

QUESTION

How many calls does `printUnique("AAB", 0)` make, and how many would `printAll` make?

ANSWER

`rest.push_back(r);` then `if (solve(rest)) return true;` then `rest.pop_back();`

After the pop, `rest` is again the other numbers, ready for the next result. Without the pop, the next result is pushed on top of the previous one and `rest` grows to `{others, r1, r2}`.

ANSWER

Copy every `nums[t]` with `t != i && t != j`. Compare **positions**, not values.

Example: `nums = {3, 3, 8, 8}`, `i = 0`, `j = 2`. Comparing values (`nums[t] != a && nums[t] != b`) would remove all four numbers; comparing positions leaves `rest = {3, 8}`.

ANSWER

Each number is multiplied by 10^6, rounded to a whole number, and the row of integers is sorted.

Scaling and rounding can map nearby values to the same integer, but nearby values can also fall into **different** rounding bins. Coarse rounding may merge distinct states and lose solutions; finer rounding may detect fewer approximate duplicates. Rounded keys do not guarantee exact state equivalence. 10^6 is a compromise.

ANSWER

Floating-point calculations can introduce small rounding errors. A result mathematically equal to 24 may be represented slightly above or below 24. The tolerance `1e-9` accepts such differences. It is a numerical policy for small inputs such as card values, not an exact test.

ANSWER

`printUnique`: **9** calls. `printAll`: **16** calls.

At the root, the second `A` is skipped (it saves 5 calls), and under `BAA` the second `A` is skipped again (it saves 2): $9 + 5 + 2 = 16$.

ANSWER

A key in `seen` proves only that the **key** was encountered before. "No solution" follows if (1) keys identify equivalent states **exactly**, and (2) a success returns `true` all the way out.

Rounded keys can collide, so (1) may fail.

A visited state has **finished**: each move removes one value, so an equal-sized state cannot be an active ancestor. This does not extend to cyclic searches.